

Spring Testing

Chapter 03 - Spring Framework

Sub-Chapter 03.10

Testing Pyramid	Unit vs Integration vs E2E concepts
JUnit 5 Basics	@Test, @BeforeEach, @AfterEach, Assertions
Mockito	@Mock, @InjectMocks, when/thenReturn, verify
@SpringBootTest	Full application-context integration test
@WebMvcTest	Controller slice test with MockMvc
@DataJpaTest	Repository slice test with H2 in-memory DB
TestContainers	Real database in Docker for tests

1 Testing Pyramid

Unit -> Integration -> E2E

The Testing Pyramid helps you decide how many tests of each type to write. Unit tests are cheap and fast - write many of them. Integration tests verify how components work together. E2E tests cover the entire application flow but are slow, so keep them few.

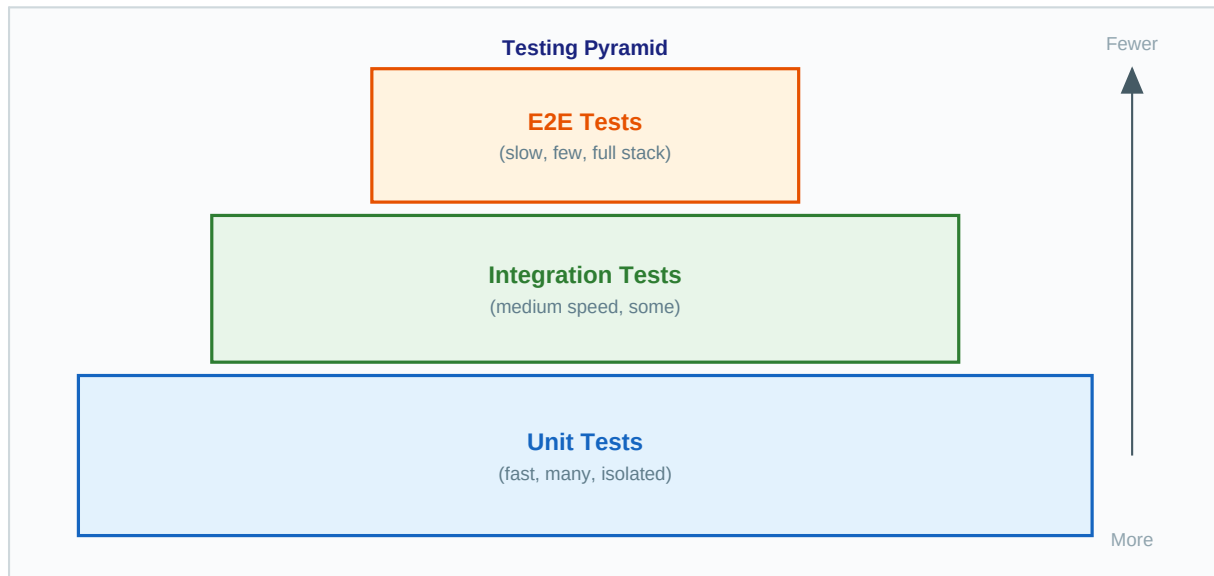


Fig 1.1 - Testing Pyramid layers

Example - simple Java concepts:

```
// Unit test - tests one class in isolation
class CalculatorTest {
    Calculator calc = new Calculator();

    @Test
    void addsTwoNumbers() {
        assertEquals(5, calc.add(2, 3));
    }
}

// Integration test - loads Spring context
@SpringBootTest
class OrderServiceIntegrationTest { ... }

// E2E test - drives the browser or HTTP client
class CheckoutE2ETest { ... }
```

Key Rules

- Unit tests: no Spring context, no DB - pure Java logic.
- Integration tests: involve 2+ components (service + repo, controller + DB).
- E2E tests: full stack; use sparingly because they are slow and fragile.
- Aim for roughly 70% unit, 20% integration, 10% E2E.

Q1. What is the Testing Pyramid?

A model suggesting you write many cheap unit tests, fewer integration tests, and very few slow E2E tests.

Q2. Why are unit tests preferred at the base?

They run in milliseconds, have no external dependencies, and give fast feedback during development.

Q3. What does an integration test verify?

That multiple components (e.g., a service and a repository) work correctly together.

Q4. What is an E2E test?

A test that exercises the full application stack, often through HTTP or a browser, from end to end.

Q5. Why should E2E tests be kept to a minimum?

They are slow, brittle (break on UI changes), and expensive to maintain compared to unit tests.

2 JUnit 5 Basics

@Test, @BeforeEach, @AfterEach, Assertions

JUnit 5 (Jupiter) is the standard Java testing framework. Annotate methods with @Test to mark them as tests. Use @BeforeEach to set up state before every test and @AfterEach to clean up. Assertions.assertEquals and friends verify outcomes.

Example:

```
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class BankAccountTest {
    BankAccount account;

    @BeforeEach
    void setUp() { account = new BankAccount(100); }

    @AfterEach
    void tearDown() { account = null; }

    @Test
    void depositIncreasesBalance() {
        account.deposit(50);
        assertEquals(150, account.getBalance());
    }

    @Test
    void withdrawThrowsWhenInsufficient() {
        assertThrows(IllegalStateException.class,
            () -> account.withdraw(200));
    }
}
```

Key Rules

- @Test marks a method as a test case - no public/void requirement in JUnit 5.
- @BeforeEach runs before EACH test - ideal for creating fresh objects.
- @AfterEach runs after EACH test - ideal for releasing resources.
- Assertions.assertEquals(expected, actual) - expected value comes FIRST.
- assertThrows(ExceptionClass, executable) verifies an exception is thrown.

Q1. How do you mark a test method in JUnit 5?

Annotate the method with @Test. The method must return void but does not need to be public.

Q2. What is @BeforeEach used for?

To run setup code before every single test method in the class - e.g., initializing objects.

Q3. How do you check that two values are equal?

Use Assertions.assertEquals(expected, actual) - or statically import it as assertEquals(expected, actual).

Q4. How do you assert that an exception is thrown?

Use assertThrows(ExceptionClass.class, () -> codeUnderTest()) which returns the thrown exception.

Q5. What is the difference between @BeforeEach and @BeforeAll?

@BeforeEach runs before every test method; @BeforeAll runs once before all tests in the class (must be static).

3 Mockito

@Mock, @InjectMocks, when/thenReturn, verify

Mockito lets you replace real dependencies with fakes (mocks). @Mock creates a mock object. @InjectMocks creates the class under test and injects the mocks into it. Use when/thenReturn to stub behaviour and verify() to check that a method was called.

Example:

```
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {

    @Mock
    OrderRepository repo;

    @InjectMocks
    OrderService service;

    @Test
    void returnsOrderById() {
        Order order = new Order(1L, "Book");
        when(repo.findById(1L)).thenReturn(Optional.of(order));

        Order result = service.getOrder(1L);

        assertEquals("Book", result.getName());
        verify(repo).findById(1L);
    }
}
```

Key Rules

- @ExtendWith(MockitoExtension.class) activates Mockito annotations.
- @Mock creates a mock - all methods return default values (null/0/false).
- @InjectMocks creates the real object and injects @Mock fields into it.
- when(mock.method(args)).thenReturn(value) stubs a method call.
- verify(mock).method(args) asserts the method was called exactly once.

Q1. What does @Mock do?

It creates a mock (fake) implementation of a class or interface so you can control its behaviour in tests.

Q2. What does @InjectMocks do?

It creates an instance of the class under test and automatically injects all @Mock fields into it.

Q3. How do you stub a method to return a specific value?

Use when(mock.someMethod(args)).thenReturn(value) before calling the method under test.

Q4. How do you verify a method was called?

Use verify(mock).someMethod(args) - this fails if the method was not called exactly once.

Q5. What happens if you call a mock method with no stub?

Mockito returns a default: null for objects, 0 for numbers, false for booleans, empty collections.

4 @SpringBootTest

Full context integration test

@SpringBootTest loads the complete Spring application context - all beans, configurations, and the embedded server (optionally). Use it when you need to test the interaction of many layers together. It is slower than slice tests because it wires the entire application.

Example:

```
@SpringBootTest(webEnvironment =
    SpringBootTest.WebEnvironment.RANDOM_PORT)
class UserApiIntegrationTest {

    @Autowired
    TestRestTemplate restTemplate;

    @Test
    void createUserReturns201() {
        UserDto dto = new UserDto("Alice", "alice@test.com");
        ResponseEntity<UserDto> res =
            restTemplate.postForEntity("/users", dto, UserDto.class);
        assertEquals(HttpStatus.CREATED, res.getStatusCode());
    }
}
```

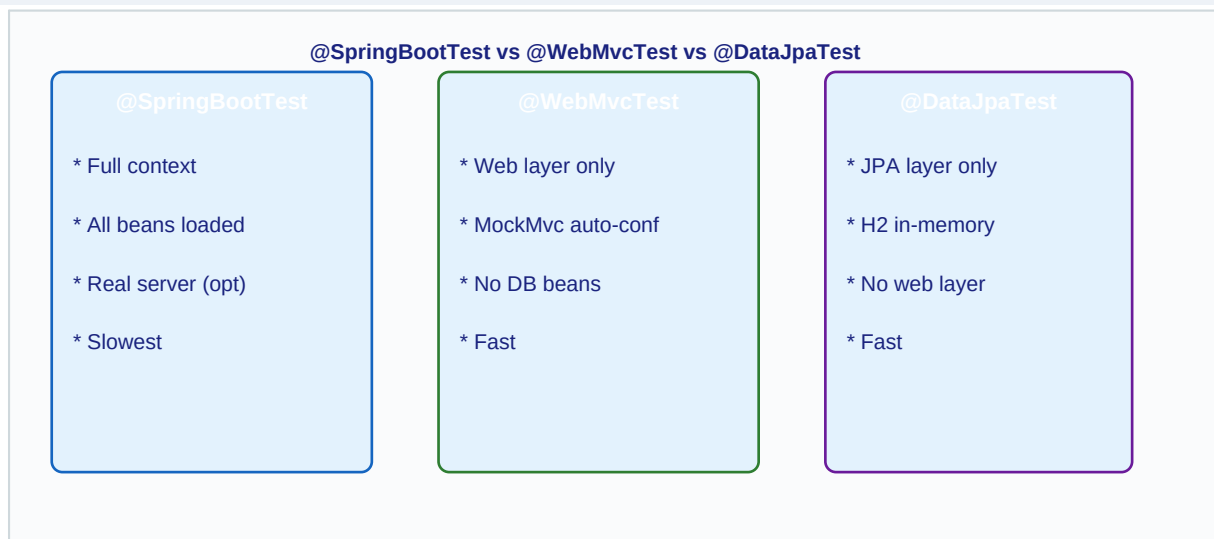


Fig 4.1 - What each test slice annotation loads

Key Rules

- @SpringBootTest loads ALL beans - use when you need full wiring.
- WebEnvironment.RANDOM_PORT starts a real server on a random port.
- WebEnvironment.MOCK (default) creates a mock servlet environment.
- Inject TestRestTemplate or WebTestClient to make HTTP calls.
- Prefer slice tests (@WebMvcTest, @DataJpaTest) for faster feedback.

Q1. What does @SpringBootTest load?

The complete Spring application context - all beans, auto-configurations, embedded server (optional).

Q2. What is WebEnvironment.RANDOM_PORT?

It starts a real embedded server on a random available port, allowing true HTTP calls with TestRestTemplate.

Q3. When should you use @SpringBootTest over slice tests?

When you need to verify the interaction of multiple layers - e.g., controller + service + repository together.

Q4. What is TestRestTemplate?

A test-friendly HTTP client provided by Spring Boot that connects to the embedded test server.

Q5. What is the main downside of @SpringBootTest?

It is slow because it loads every bean in the application - prefer slice tests for unit-level checks.

5 @WebMvcTest

Slice test for controllers, MockMvc

@WebMvcTest loads only the web layer: controllers, filters, and MVC configuration. The service and repository beans are NOT loaded - you mock them with @MockBean. MockMvc lets you send HTTP requests and inspect responses without starting a real server.

Example:

```
@WebMvcTest(ProductController.class)
class ProductControllerTest {

    @Autowired
    MockMvc mvc;

    @MockBean
    ProductService service;

    @Test
    void getProductReturns200() throws Exception {
        when(service.findById(1L))
            .thenReturn(new ProductDto(1L, "Laptop"));

        mvc.perform(get("/products/1"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.name").value("Laptop"));
    }
}
```

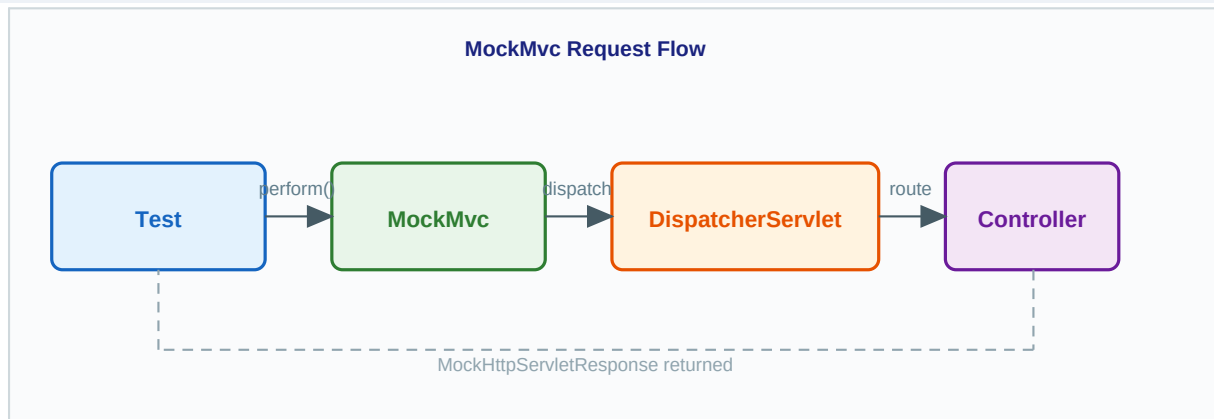


Fig 5.1 - MockMvc request dispatching flow

Key Rules

- @WebMvcTest(Controller.class) loads only that controller and web beans.
- @MockBean replaces a real Spring bean with a Mockito mock in the context.
- MockMvc.perform() sends a request; andExpect() asserts the response.
- status().isOk() checks HTTP 200; jsonPath() inspects JSON body fields.
- No real HTTP server is started - tests run in-process and are fast.

Q1. What does @WebMvcTest load?

Only the web layer: controllers, filters, @ControllerAdvice, and MVC auto-configuration. No service or repo beans.

Q2. Why do you need @MockBean inside a @WebMvcTest?

Because service/repository beans are not loaded; @MockBean creates mocks for the dependencies the controller needs.

Q3. How do you send a GET request with MockMvc?

Call `mvc.perform(get("/path"))` and chain `andExpect()` calls to assert status, headers, and body.

Q4. What is the difference between @Mock and @MockBean?

@Mock is a plain Mockito mock not registered in the Spring context; @MockBean registers the mock as a Spring bean.

Q5. What is the advantage of @WebMvcTest over @SpringBootTest for controller tests?

It is much faster because only the web slice is loaded, not the entire application context.

6 @DataJpaTest

Slice test for repositories, H2 in-memory

@DataJpaTest loads only the JPA layer: entity classes, repositories, and an in-memory H2 database. It does NOT load controllers or services. Each test runs inside a transaction that is rolled back after the test, keeping the database clean.

Example:

```
@DataJpaTest
class CustomerRepositoryTest {

    @Autowired
    CustomerRepository repo;

    @Test
    void savesAndFindsCustomer() {
        Customer c = new Customer("Bob", "bob@test.com");
        repo.save(c);

        Optional<Customer> found =
            repo.findByEmail("bob@test.com");

        assertTrue(found.isPresent());
        assertEquals("Bob", found.get().getName());
    }
}
```

Key Rules

- @DataJpaTest configures H2 in-memory DB, JPA, and Hibernate automatically.
- Each test method is wrapped in a transaction rolled back at the end.
- Only @Repository, @Entity, and JPA config beans are loaded.
- No web layer, no service beans - keeps tests focused and fast.
- Use @AutoConfigureTestDatabase(replace=NONE) to test against a real DB.

Q1. What does @DataJpaTest configure automatically?

An in-memory H2 database, JPA/Hibernate settings, and your Spring Data repositories.

Q2. Why does @DataJpaTest use transaction rollback by default?

To keep tests isolated - changes made in one test are undone so the next test starts with a clean state.

Q3. What beans are NOT loaded with @DataJpaTest?

Web layer beans (controllers) and service-layer beans are excluded - only the JPA slice is loaded.

Q4. How do you use a real database instead of H2 in @DataJpaTest?

Add @AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE) to keep your real datasource.

Q5. What is the advantage of @DataJpaTest over @SpringBootTest for repository tests?

It is much faster - only the persistence layer is loaded and an embedded DB is configured automatically.

7 TestContainers

Real database in Docker for tests

TestContainers starts real Docker containers (PostgreSQL, MySQL, Redis, etc.) during your tests. This lets you test against the exact same database your production code uses, avoiding H2 compatibility surprises. The container starts before tests and stops when they finish.

Example:

```
@SpringBootTest
@Testcontainers
class OrderRepoContainerTest {

    @Container
    static PostgreSQLContainer<?> postgres =
        new PostgreSQLContainer<>("postgres:15")
            .withDatabaseName("testdb")
            .withUsername("test")
            .withPassword("test");

    @DynamicPropertySource
    static void props(DynamicPropertyRegistry r) {
        r.add("spring.datasource.url", postgres::getJdbcUrl);
        r.add("spring.datasource.username", postgres::getUsername);
        r.add("spring.datasource.password", postgres::getPassword);
    }

    @Autowired OrderRepository repo;

    @Test
    void savesOrder() {
        repo.save(new Order("Book"));
        assertEquals(1, repo.count());
    }
}
```

Key Rules

- @Testcontainers enables the TestContainers JUnit 5 extension.
- @Container marks a container field - static = one container for all tests.
- @DynamicPropertySource feeds the container's URL/credentials to Spring.
- Requires Docker to be installed and running on the machine.
- Use for production-parity tests; rely on H2/@DataJpaTest for speed.

Q1. What problem does TestContainers solve?

It provides a real database in Docker so tests run against the same DB engine as production, avoiding H2 quirks.

Q2. What does @Container do?

It marks a field as a managed Docker container - TestContainers starts it before tests and stops it after.

Q3. What is @DynamicPropertySource used for?

It overrides Spring properties at runtime, pointing datasource URL/credentials to the running Docker container.

Q4. What is the difference between a static and an instance @Container?

Static: one container shared across all tests in the class (faster). Instance: a new container per test (cleaner).

Q5. When should you prefer TestContainers over @DataJpaTest with H2?

When your code uses database-specific SQL, JSON columns, or stored procedures that H2 does not support.